什么是真正的Agent

- 来源: https://class.imooc.com/lesson/5289#mid=105147
- 阶段: 阶段六: 【智能体】从单体到多体
- 周: 第11周 Agent：智能体架构
- 模块: 什么是真正的Agent
- 课节: 图文 1-1 什么是真正的Agent

1-1 什么是真正的Agent

什么是真正的Agent

> [!TIP] 阅读指南 在之前LangChain章节中，我们提到过Agent——智能体。但我们并没有深入讨论这个话题。这一节，而是回到一条基础的问题线上： Agent 到底是什么？ 它和“聊天机器人”“工具调用”有本质区别吗？ 学术界眼中的 Agent，和工业界的 Agent 产品是一回事吗？ 只有把这些问题理清，后面谈架构、记忆、推理模式、多 Agent 协作时，你才不会迷失在各种名词和框架细节里。

从聊天机器人到智能体的认知跃迁

先用一个具体的对比，帮你把脑子里的两个形象区分开来。

"会聊天的模型"：增强版搜索引擎

大多数人第一次接触 ChatGPT 时，直觉上会把它理解成一个“超强的问答机器人”，它的典型特征是：

- 输入—输出是一次性的：你问一句，它答一句；

输入—输出是一次性的

- 没有持续的目标：不会自己设定要完成什么，只是被动回应问题；

没有持续的目标

- 几乎不主动行动：你让它写代码、给建议、出方案，它就在“文字空间”里给你一堆文本；
- 记忆很短暂：一旦换了对话，之前的所有上下文、约定、决策都要重来。

记忆很短暂

这种形态，本质上还是一个“超级回答机”。

“能自己干事的 Agent”：从回答到"达成目标"

想象这样一个场景：

> 你跟一个旅行助手说：“帮我规划下周去东京的 3 日行程，要求控制预算，帮我订好机票和酒店，并把行程同步到日历里。” 然后你关掉聊天窗口，去忙别的事。 几个小时后，它给你发来一封总结邮件：机票已订、酒店已订、日历已更新、还顺手帮你查了天气并调整了行程。

在这个故事里，这个“旅行助手”明显不再只是一个“问一句答一句”的聊天机器人，而更像一个“可以托付任务的智能助理”：

- 它理解了你的目标（不是“回答一个问题”，而是“完成一次出行的整体安排”）

理解了你的目标

- 它会分解任务（查航班 → 比价 → 选酒店 → 生成日程 → 调用日历 API）
- 它会在过程中自主决策与调整（预算超标就换方案，不可行就尝试别的）

自主决策与调整

持续修正行为

这里的核心区别在于： 聊天机器人是“对话优先”的，Agent 是“目标优先”的。

- 对聊天机器人来说，对话本身就是目的：好好回答你的问题；
- 对 Agent 来说，对话只是手段之一：真正的目的，是帮你把事办成。

帮你把事办成

这就是从“聊天机器人”到“智能体”的第一步认知跃迁： 从“对话产品”到“面向目标的智能决策体”。

Agent 的学术定义：从"感知—思考—行动"的循环看世界

如果从学术视角看，Agent 并不是一个新词。早在传统 AI 时代（远早于大模型），"智能体（Intelligent Agent）"就有一套相对成熟的框架。所以，我们必须有一个认识：Agent并不独属于LLM，也不是因为LLM才有了Agent。

教科书式的定义

在经典 AI 教科书里，一个常见的定义是：

> Agent 是一个能够感知环境（Perception），并基于感知做出行动（Action）的实体。

这个定义听起来很抽象，我们把它拆成几个关键要素：

- 环境（Environment）：Agent 所在的世界 对游戏 AI 来说，是游戏世界； 对推荐系统里的智能体来说，是用户行为构成的信息世界； 对我们这一章要讲的 LLM Agent 来说，环境可以是： 一段对话历史 一个文件系统 一组可调用的工具和 API
- 感知（Perception）：从环境中获取信息的过程 游戏角色通过视野/传感器感知敌人位置； 机器人通过摄像头和传感器感知周围障碍物； LLM Agent 通过“读上下文” “查看工具返回结果” “访问记忆”来感知世界。
- 动作（Action）：改变环境或自身状态的行为 游戏角色移动、攻击、躲避； 机器人抓取物体、移动位置； LLM Agent 调用一个工具、写入一条记忆、向用户发消息、创建一个新的子任务。

在这个框架下，Agent 不再被看作“一个函数调用”或“一个 API”，而是一个持续运行、不断循环的过程：

- 感知（Perceive）：接收当前环境的状态；

感知（Perceive）

- 决策（Decide / Think）：基于目标和历史经验做出判断；

决策（Decide / Think）

- 行动（Act）：执行某个动作（包括对话、工具调用等）；

行动（Act）

- 环境变化：世界因行动而改变；

环境变化

- 回到第 1 步，继续循环。

工业界的 Agent 实践：从"流程自动化"到"智能决策体"

学术界给了我们抽象框架，工业界则给了我们一个又一个落地场景。 这两者有时会有偏差：

- 学术界关心的是理论完备性和可证性；

- 工业界更在意的是：能不能落地？成本多少？效果有没有显著提升？

能不能落地？成本多少？效果有没有显著提升？

早期的"伪 Agent"时代：规则+流程自动化

在大模型之前，“Agent”这个词就已经出现过很多次，只不过当时的实现方式主要是：

- 规则引擎 + 流程自动化 根据用户输入匹配意图（Intents） 用 if/else 或工作流引擎触发预定义流程 每一步都是人类开发者事先设计好的固定路径

这类系统很多公司叫它：

- “智能客服机器人”
- “虚拟助理”
- “智能流程引擎”

它们确实有“感知—行动”的轮廓，但“思考”部分基本被设计成了硬编码规则。一旦场景稍微复杂一点、用户说话方式稍微多样一点，整个系统就显得非常僵硬。

大模型加持后的转折：从"语义理解"到"通用问题求解器"

当大模型引入之后，有两个关键变化：

- 自然语言理解能力大幅提升：

自然语言理解能力大幅提升

早期没有LLM前，Agent最大的难题是很难理解用户的意图，因为机器本身不懂物理世界也不懂人类的语言，这种情况下我们往往会用标签、特征这些看起来比较笨的方法来让机器理解。

但有了LLM后，Agent不再需要费尽心思想各种意图标签、特征工程，系统可以直接“读懂”用户在说什么。大模型开始具备“在未知任务上也能给出合理方案”的能力——这就是我们在之前讨论过的“涌现能力”。

- Agent 的运行模式随之改变：

Agent 的运行模式随之改变

在这个基础上，工业界的 Agent 实践出现了新形态：

- 用大模型做 目标理解和计划生成；

目标理解和计划生成

- 用工具调用 / API / 函数调用 去 落实行动；

落实行动

- 用向量数据库和外部知识库做 长期记忆；

长期记忆

- 用工作流编排 / LangGraph / 自研框架做 多轮决策和状态管理。

多轮决策和状态管理

于是，你在各种产品和论文中会看到：

- “AI 代码助手 Agent”
- “AI 运维 Agent”
- “AI 财务分析 Agent”
- “AI 游戏 NPC Agent”
- “多 Agent 协作平台 / Agentic Workflow / Agentic RAG …”

它们背后的共同点是：不再把大模型当成一个"单轮问答机"，而是把它放进一套"感知—思考—行动"的循环中去。

工业界的妥协与工程化现实

不过，需要特别提醒的是：工业界的 Agent，与教科书里的“理想 Agent”，之间有几层重要的工程现实：

- 安全与可控性优先：企业不会允许一个 Agent 随意删库、乱发邮件、乱操作生产系统。因此工业界会通过：
- 严格的权限控制
- 人类在环（Human-in-the-loop）
- 沙箱环境
- 审批流和回滚机制 把 Agent 的行为限制在可控的“轨道”上。
- 可靠性与稳定性约束：纯粹依赖推理能力和“即兴发挥”的 Agent，在生产环境里很危险。 所以工程上会加入：
- 明确的工具调用协议
- 错误重试机制
- 退避策略和兜底流程

- 可观测性和日志系统 让整个系统在出错时可诊断、可修复。
- 成本与收益权衡：一个“聪明但昂贵”的 Agent，往往难以大规模落地。工业界经常会做的一件事是：
- 把 Agent 用在“高价值 + 低频”的场景（比如复杂决策、长周期任务）；
- 在高频、简单、可规则化的部分，仍然使用传统系统（规则引擎、工作流）。

正是这些工程上的"约束"和"妥协"，让今天你在框架里看到的 Agent，是一种兼顾智能性与可控性的工程产物。

Agent 的核心特征：不只是"会调用工具"

现在，我们可以尝试给出一个稍微务实、又兼顾理论的总结： 在本书里，当我们说"真正的 Agent"，至少包含下面这些特征。

目标导向（Goal-oriented）

不再只是“回答问题”，而是围绕目标组织行为；同时，能够从自然语言中抽取和澄清目标：

- “帮我写一封辞职信”
- “帮我把这个项目改成使用 LangGraph”
- “帮我复盘一下这次线上事故，并给出整改方案”

更理想的 Agent，还能够在执行过程中不断更新或细化目标：

- 发现目标不清晰时向你发问；
- 发现原设目标不可行时，提出备选目标。

持续性（Persistence）

- 有跨轮次对话的记忆：不是每轮都从零开始；

跨轮次对话的记忆

- 有跨任务的经验积累：在理想形态下，做同一类任务会越做越好；

跨任务的经验积累

- 有内部状态：不仅记得“你说过什么”，还记得“自己做过什么、学到什么”。

内部状态

在工程上，这意味着我们需要给 Agent 设计：

- 短期记忆：当前任务的上下文；
- 长期记忆：跨任务的知识、偏好、经验；
- 元记忆：关于“自己是谁、应该如何行动”的更高层规则。

可行动（Actionable）

Agent 不仅能“说”，还能做，比如：

- 调用 HTTP API；
- 读写数据库；
- 操作文件系统；
- 控制外部服务（例如 CI/CD、工单系统、日历、邮件）。

通常我们会用 Function Calling、工具调用、MCP、插件等机制，把这些动作暴露给 Agent。

可感知（Perceptive）

它能主动去获取信息，比如：

- 读文档、查数据库、调用搜索；
- 查看某个任务的状态；
- 访问其他 Agent 的输出。

它可以对“环境变化”做出响应，比如：

- 发现 API 报错时调整策略；
- 发现数据更新后重新计算；
- 发现用户需求改变后重新规划。

这里“感知”的关键在于：Agent 不只被动等待输入，而是会主动“去看一眼世界”。

可解释与可控（Controllable & Interpretable）

一个能够在生产环境落地的 Agent，必须具备一定的可解释性和可控性：

- 我们需要知道：它为什么做出这个决策？
- 我们可以：在关键步骤把人类拉进来“拍板”；
- 我们需要手段：在出现异常行为时及时“拉手刹”。

因此现代的 Agent 系统里，会大量使用：

- Chain-of-Thought 推理链 / 思维树（Tree-of-Thought）来让决策过程可追踪；
- 可视化的状态图和执行图（如 LangGraph 的状态机）；
- 对关键动作的审批和审计日志。

当一个系统同时具备目标导向、持续性、可行动、可感知、可解释与可控这些特征时，我们才有理由说：

“这不只是一个高级聊天机器人，而是一个真正意义上的 Agent。”

一个完整的例子：游戏 NPC vs LLM Agent

游戏里的 NPC（Non-Player Character，非玩家角色）是理解 Agent 的一个极好比喻。

传统 NPC：预设行为脚本

在传统游戏里，一个 NPC 的行为大概是这样被设计的：

- 在固定位置巡逻；
- 检测到玩家进入视野范围 → 切换为追击状态；
- 玩家离开一定距离 → 放弃追击，返回巡逻路径；
- 血量低于某个阈值 → 逃跑或寻求治疗。

这些行为通常用有限状态机（FSM）或行为树（Behavior Tree）实现，本质上是高度规则化的反应式 Agent：

- 感知有限。只能感知少量预定义事件。
- 决策空间有限，只在少数几种状态之间切换。这也是很多游戏里当玩家做出一些超出预定事件时，NPC会出现BUG的原因，因为他无法对没有编程的行为做出正确的反应。
- 几乎不会“出人意料”——这既是优点，也是局限。

你在很多早期的单机游戏里，可能都体验过这种 NPC：

打着打着，你会开始“计算”它的 AI—— 知道它在什么时候会转身、何时会触发某种动作，甚至可以设计“卡 AI”的套路。这就是有限规则所带来的弊端，玩家完全可以根据反复的测试找到NPC的规律。

如果用 LLM 来做 NPC Agent，会有什么不同?

假设我们用大模型来驱动一个开放世界 RPG 游戏里的 NPC，比如一个“冒险者公会接待员”。 它的世界可能是这样的：

- 环境： 当前游戏世界的状态（时间、天气、地区局势） 玩家任务进度和历史行为 公会的规章制度和资源状态（任务池、可用奖励等）
- 感知： 玩家当前对话内容； 游戏后端系统提供的 API（任务数据库、玩家状态查询等）返回结果。
- 行动： 用自然语言与玩家对话； 给玩家分配任务、修改任务状态； 在游戏世界里触发某些事件（刷新怪物、解锁区域、发放奖励）。

在这种设定下，这个 NPC Agent 可以做一些传统 NPC 很难做到的事情：

- 根据玩家风格动态设计任务： 如果玩家偏爱探索，就多给探索任务； 如果玩家喜欢战斗，就多给战斗任务。
- 在对话中展现出一致的人设与记忆： 记得你上次搞砸了某个任务，会在这次对话里调侃你； 会根据你之前对某个势力的态度，调整给你的剧情线。
- 在复杂局势下做出合理的“创作级”决策： NPC 不再只是执行几个预设剧情分支，而是可以“即兴编排”符合世界观的新剧情。

这个 NPC，已经非常接近我们在本书中要讨论的“真正的 Agent”：

- 它不是执行一条流程，而是在一个丰富的世界中持续生存；

在一个丰富的世界中持续生存

- 它的行为不仅要“正确”，还要“有趣” “有一致性”；
- 它既要满足游戏设计师的意图，又要对玩家的行为做出灵活反应。

这样的NPC Agent可以让你无论玩多少次游戏，每一次可能都是不同的体验。

> 游戏业界的AI NPC实践 其实现在已经有很多游戏在尝试用LLM来做真正的Agentic NPC（智能化的NPC），虽然还面临技术挑战，但已有不少成功案例：

> 育碧的项目允许玩家通过自然语言指挥AI队友，这些NPC能够理解战术指令并在动态战场中做出相应行动；Meta也在其Worlds平台推出了LLM驱动的NPC工具，让开发者能够快速创建具备动态对话能力的角色。 这些实践印证了本章讨论的核心观点：Agent不只是一个技术概念，而是正在各个领域落地的智能化方向。

回到现实中的 LLM Agent

你可能不会真的去做一个开放世界 RPG 的 NPC Agent，但实际工程中，我们做的很多 Agent，本质上都与这个 NPC 很像。

- 一个“AI 代码助手 Agent”： 以你的项目代码库和工作记录为环境； 通过读取文件、调用构建系统、分析日志来感知； 通过生成代码、提交 MR、写评审意见来行动。
- 一个“企业运维 Agent”： 以监控系统、服务拓扑、告警信息为环境； 通过查询日志、检查服务依赖、访问 APM 工具感知问题； 通过重启服务、调整配置、创建工单、通知值班同学来行动。

它们都符合同一套抽象：在一个复杂环境中，围绕目标，持续进行"感知—思考—行动"的循环。

冷知识：Qoder就是一个典型的 Agent

说了这么多抽象的概念，最后不妨用一个你每天都在使用的对象来收个尾—— 其实你现在正在使用的 Qoder，本身就是一个非常典型的智能 Agent。

如果我们用本节的框架来“解剖”一下 Qoder，你会发现几乎所有 Agent 的关键特征都在它身上：

目标导向：

- 你并不是在和它“闲聊”，而是在不断下达一个个任务目标： “帮我实现一个二叉树层序遍历” “重构这一段代码，让它更易维护” “根据这个接口文档写一组单元测试”
- 感知环境： 对 Qoder 来说，它的“世界”包括： 你的代码仓库（项目结构、依赖关系、已有模块）； 你每一次的开发指令和评论（比如： “这个函数太长了”）； 每当你发出新指令，它都会先“观察环境”：读取相关代码文件、理解当前改动的上下文、回顾你之前的约定，然后才开始给出修改建议或生成新的实现。
- 思考与决策： Qoder 并不是简单地“把你说的话翻译成另一段话”，而是要在内部完成一套决策过程： 先推理：你真正想要的是什么？是修一个 bug，还是做一次重构，还是设计一套新接口？ 再规划：这次改动大概要分为几个步骤？是先补测试还是先改实现？要不要先在一个小函数上试验？ 最后落笔：根据规划一步步生成代码改动方案（重命名、抽函数、拆模块等），并在需要时附上解释。 这个过程本质上就是一个典型的“感知—思考—行动”中的“思考”阶段，只不过它发生在大模型的内部推理空间里。
- 可行动：通过工具影响“外部世界”： 在你的开发工作流里，Qoder 不只是“给出一段代码片段”这么简单，它还可以： 直接修改本地代码文件（根据你的确认，把某个函数重写、插入日志）； 调用构建 / 测试工具，帮你跑单元测试或静态检查； 换句话说，它已经不仅仅活在聊天窗口里，而是在通过工具调用，真实地改变你的代码库状态和开发流程——这正是 Agent 相比普通聊天机器人的关键跃迁。
- 记忆与一致性： 在一次完整的开发会话中，Qoder 会持续记住： 当前正在修改的是哪一块业务代码、有哪些关联模块； 你对某些风格（比如“能否使用魔法数” “异常要不要层层抛出”）的偏好； 你之前已经在哪些文件里做过哪些约定（例如日志格式、错误码规范），避免前后不一致。 在

更长的时间尺度上，它还可以借助“外部记忆”机制（例如你的代码规范文档、架构说明、项目 README）来保持整个代码库的风格与约束一致。

- 可解释与可控： 你可以随时打断它，要求它：“解释一下你刚才为什么这么改代码？” 你也可以对它的“行动权限”加以约束： 只让它在某几个模块或某个分支里动手； 对关键代码（比如安全敏感逻辑、结算逻辑）只允许它给出建议，由你亲自确认和提交。 这实际上就是对 Agent 的可控性设计：让它有足够的主动性，但始终处在你的代码评审和版本控制体系之下。

从这些角度看，现在并不是"在用一个 AI 工具写代码"，而是在和一个 Agent一起完成一个项目——这本身，就很有趣。

下节预告

现在理解了 Agent 的本质特征，但可能会好奇：Agent 并不是一夜之间出现的，它是如何从最初的"单轮对话"一步步演化成今天这样强大的智能体的？

从 ChatGPT 最初只能"聊天"，到后来可以"调用工具"，再到如今能够"自主规划并完成复杂任务"——这背后经历了哪些关键的技术跃迁？每一次演进又解决了什么核心问题？

下一节，将沿着 Agent 的进化时间线，完整理解从 Function Calling 到真正的自主 Agent 的演进路径。可以看到，原来今天的 Agent 能力，是一个个技术突破叠加起来的结果。

■ 学点英语

中文	English	音标	说明
智能体	Agent	/ˈeɪdʒənt/	能够感知环境并基于感知做出行动的实体
感知	Perception	/pəˈsepʃn/	Agent从环境中获取信息的过程
行动	Action	/ˈækʃn/	Agent改变环境或自身状态的行为
目标导向	Goal-oriented	/goʊl ˈɔːrientɪd/	Agent围绕目标组织行为，而非仅仅回答问题
持续性	Persistence	/pərˈsɪstəns/	Agent保持跨轮次记忆和内部状态的能力
感知-思考-行动	PTA (Perceive-Think-Act)	/pɜːrˈsiːv θɪŋk ækt/	Agent运行的核心循环框架